

Function-Oriented Software Design

Introduction

- Function-oriented design techniques are popular:
 - Used in software development organizations, even today.
- Function-oriented design techniques:
 - Start with functional requirements specified in SRS document.
- During the design process:
 - High-level functions are successively decomposed into more detailed functions.
- Successive decomposition of high-level functions into more detailed functions known as Top-down decomposition.
 - Finally detailed functions are mapped to a module structure.
- SA/SD is a very widely used methodology for system decomposition.

Top-Down Decomposition

- High-level system functionality is decomposed into sub-functions
- Each level introduces more detail
- Final outcome: mapping functions to program modules
- Promotes clarity, traceability, and modularity

Why Study Function-Oriented Design?

- Many legacy and mission-critical systems are function-oriented
- Banking, telecom, embedded, ERP, and government systems still rely on it
- Understanding SA/SD improves:
 - Requirements engineering
 - System decomposition skills
 - Architectural reasoning
- Modern methods evolved from these foundations

SA / SD (Structured Analysis/Structured Design)

- SA/SD technique used to perform high-level design.
- SA/SD methodology:
 - has essential features of several important function-oriented design methodologies
 - If you need to use any specific design methodology later on, you can do so easily with small additional effort.
- SA/SD technique draws heavily from the following methodologies:
 - Constantine and Yourdon's methodology
 - Hatley and Pirbhai's methodology
 - Gane and Sarson's methodology
 - DeMarco and Yourdon's methodology

Position of SA/SD in Modern SE

- SA/SD focuses on **logical design**, not technology
- Complements agile and iterative development
- Often used implicitly during:
 - Product discovery
 - Domain modeling
 - API boundary identification

Overview of SA/SD Methodology

- SA/SD consists of two distinct activities:
 - Structured Analysis (SA)
 - Structured Design (SD)
- During Structured Analysis:
 - Functional decomposition takes place as follows.
 - Each function is analyzed - Hierarchically decomposed into more detailed functions.
 - Simultaneous decomposition of high-level data into more detailed data.
 - Transforms a textual problem description into a graphic model.
 - Done using data flow diagrams (DFDs) (or bubble charts).
 - DFDs graphically represent the results of structured analysis.
- During Structured Design:
 - System module structure is formalized.
 - All the functions represented in the DFD mapped to a module structure (aka module structure chart).

Structured Analysis

- Performs hierarchical functional decomposition
- Simultaneously refines data structures
- Converts textual problem statements into graphical models
- Primary modelling tool: **Data Flow Diagrams (DFDs)**

Structured Design

- Maps DFD functions to software modules
- Produces **structure charts**
- Focuses on:
 - Module hierarchy
 - Module interactions
 - Data interfaces

SA vs. SD

- Purpose of SA:
 - Capture the detailed structure of the system as the user views it.
 - Create a logical model using DFDs
 - Uses problem-domain terminology
- Purpose of SD:
 - Convert analysis model into implementation-oriented design
 - Produce structure charts
 - Targets programmers and architects
 - Uses technical abstractions
- The results of structured analysis can be easily understood even by ordinary customers:
 - Does not require technical knowledge.
 - Directly represents customer's perception of the problem.
 - Uses customer terminology for naming different functions & data.
- The results of structured analysis can be reviewed by customers:
 - To check whether it captures all their requirements.

Structured Analysis: Industry Perspective

- Equivalent modern activities:
 - Business process modeling
 - Event storming (conceptual level)
 - System context mapping
- Strength: forces clarity before architecture decisions
- Weakness: not suitable for UI or real-time behavior modeling

Data Flow Diagrams in Modern Systems

- DFDs naturally map to:
 - REST API request/response flows
 - Data pipelines
 - ETL and batch systems
 - Microservice interactions (logical view)
- Particularly effective for:
 - Data-centric systems
 - Compliance-heavy domains

Understanding Data Flow Diagrams

- DFD is an elegant modelling technique:
 - Useful not only to represent the results of structured analysis.
 - Applicable to other areas also:
 - e.g. for showing the flow of documents or items in an organization,
- DFD technique is very popular technique.
 - It is powerful and yet simple to understand and use.
- DFD is a hierarchical graphical model.
 - Shows the different functions (or processes) of the system, and
 - Data interchange among the processes.
- Useful to consider each function as a processing station.
 - Each function consumes some input data.
 - Produces some output data.

DFD Philosophy

- Each process consumes input data
- Each process produces output data
- DFDs represent **data movement**, not control flow
- Execution order is intentionally ignored

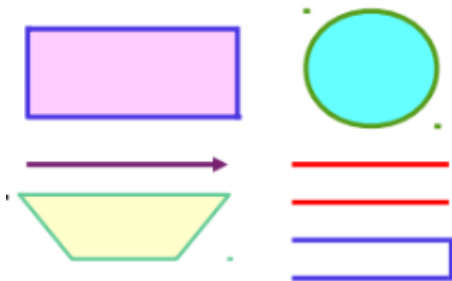
Data Flow Diagrams (DFDs)

- A DFD model:
 - Uses limited types of symbols.
 - Simple set of rules
 - Easy to understand - It is a hierarchical model.
- In a hierarchical model,
 - We start with a very simple and abstract model of a system,
 - Details are slowly introduced through the hierarchies.

Data Flow Diagrams (DFDs)

- Graphical representation of system functionality
- Shows:
 - Processes
 - Data flows
 - Data stores
 - External entities
 - System Outputs
- Hierarchical and incremental in nature

DFD Symbols



- **External Entity:** Rectangle
- **Process:** Circle (bubble)
- **Data Flow:** Directed arrow
- **Data Store:** Parallel lines, or one side (left/right) open rectangle
- **Output:** Parallelogram/Trapezium

Elements of DFD

• External Entity Symbol

- Represented using a Rectangle
- External entities are real physical entities - input data to the system or consume data produced by the system.
- Sometimes aka terminator, source, or sink.

• Function Symbol

- Represented using a Circle/Oval.
- Aka a process / bubble / transform.
- Bubbles are annotated with corresponding function names.
- Functions represent some activity - Function names should be verbs.

• Data-flow Symbol

- A directed arc or line.
- Represents data flow in the direction of the arrow.
- Data flow symbols are annotated with names of data they carry.

• Data Store Symbol

- Represented using a pair of parallel lines, or one side (left/right) open rectangle
- Represents a logical file:
 - A logical file can be a data structure, or a physical file on disk, or a table in database, or something similar.
 - Each data store is connected to a process by means of a data flow symbol.
- Direction of data flow arrow shows whether data is being read from or written into it.
- An arrow into or out of a data store:
 - Implicitly represents the entire data of that data store
 - Arrows connecting to a data store need not be annotated with any data name.

• Output Symbol

- Represented using a Trapezium or Parallelogram.
- Output produced by the system

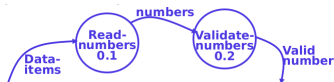
Family of DFDs (Hierarchical Model)

- Context Diagram (Aka Level 0 DFD)
 - Highest level abstraction of the system
 - Entire system shown as a single process (Bubble 0)
 - Shows:
 - External entities
 - Input and Output data
- Level 1 DFD
 - Decomposes the context diagram
 - Each major function becomes a process or a bubble
 - Shows data interaction among high-level functions
- Higher Level DFDs
 - Each bubble may be further decomposed
 - Decomposition rules:
 - 3 to 7 bubbles per level
 - Avoid excessive or trivial decomposition
 - Stop when function can be described by a simple algorithm

DFD Operations

- Synchronous Operation

- If two bubbles are directly connected by a data flow arrow, they are said to be synchronous.



- Asynchronous Operation

- If two bubbles are connected via a data store, they are not synchronous.



- No two data stores are connected as there cannot be any dataflow without any process/operation.

How is Structured Analysis (SA) Performed?

- Initially represent the software at the most abstract level:
 - Called the Context diagram.
 - The entire system is represented as a single bubble,
 - This bubble is labelled according to the main function of the system.
- A context diagram shows:
 - **Data input to the system**, **Output data generated by the system**, and **External entities**.
- Context diagram captures:
 - Various entities external to the system and interacting with it.
 - Data flow occurring between the system and the external entities.
- The context diagram is also called as the level 0 DFD.
- Establishes context of the system, & is used to represent - **Data sources and Data sinks**.

Game Software: Context Diagram

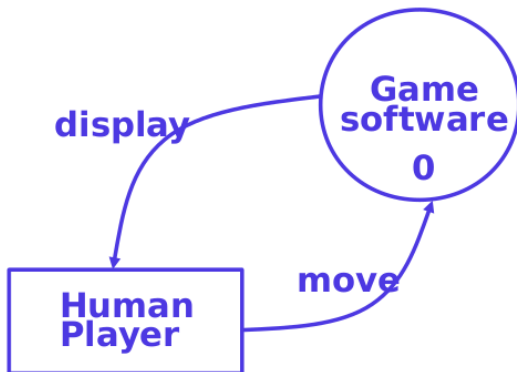
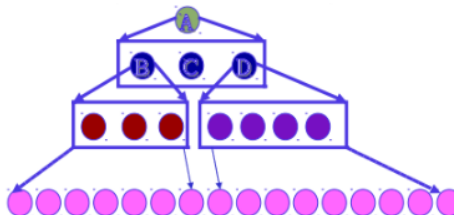


Figure: Level 0 DFD

How to construct a Level 1 and up DFDs ?

- Level 1 DFD
 - Examine the SRS / Requirements document.
 - Represent each high-level function as a bubble.
 - Represent data input to every high-level function.
 - Represent data output from every high-label function.
- Higher Level DFDs
 - Each high-level function is separately decomposed into subfunctions. These are represented as subsequent level DFDs.
 - Identify the subfunctions of the function.
 - Identify the data input to each subfunction
 - Identify the data output from each subfunction



Decomposition

- Decomposition of a bubble - Also called factoring or exploding.
- Each bubble is decomposed to between 3 to 7 bubbles.
- Too few bubbles make decomposition superfluous:
 - If a bubble is decomposed to just one or two bubbles, then this decomposition is redundant.
- Too many bubbles:
 - More than 7 bubbles at any level of a DFD.
 - Make the DFD model hard to understand.
- Decompose How Long?
 - Decomposition of a bubble should be carried on until: A level at which the function of the bubble can be described using a simple algorithm.

Level 1 DFD as Service Decomposition

- Level 1 bubbles often align with:
 - Microservices
 - Major backend services
 - Business capabilities
- Helps avoid
 - God services - is a service that knows or does too much.
 - Excessive coupling

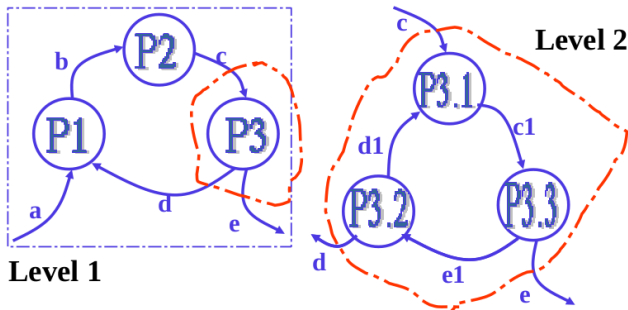
Numbering of Bubbles

- Number the processes/bubbles in a DFD:
 - Numbers help in uniquely identifying any bubble from its bubble number.
- The bubble at context level is assigned number 0.
- Bubbles at level 1:
 - Numbered 0.1, 0.2, 0.3, etc
 - Alternatively, 1, 2, 3, etc. as the prefix of 0 in reference to context diagram is assumed and hence ignored.
- Decomposition of process x gives: $x.1$, $x.2$, $x.3...$
- Ensures traceability across levels.

Balancing a DFD

- Inputs and outputs of a bubble must match its decomposition.
Alternatively, data flowing into or out of a bubble must match the data flows at the next level of DFD.
- No new data flows may appear or disappear
- Ensures consistency across DFD levels

Example: Balancing a DFD ...cont'd



- In the level 1 of the DFD, data item c flows into the bubble P3 and the data item d and e flow out.
- In the next level, bubble P3 is decomposed.
 - The decomposition is balanced as data item c flows into the level 2 diagram and d and e flow out.

Balancing DFDs and API Contracts

- Balanced DFDs resemble stable API contracts
- Prevents:
 - Hidden dependencies
 - Data leakage across layers
- Unbalanced DFDs often signal poor service boundaries

Data Dictionary

- A DFD is always accompanied by a data dictionary.
- A data dictionary lists all data items appearing in a DFD:
 - Definition of all composite data items in terms of their component data items.
 - All data names along with the purpose of the data items.
- For example, a data dictionary entry may be:
 - $\text{grossPay} = \text{regularPay} + \text{overtimePay}$
- Significance of Data Dictionary
 - Provides a standard vocabulary and terminology for all data in a project.
 - A consistent vocabulary for data is very important
 - Different team members tend to use different terms to refer to the same data, and it causes unnecessary confusion.
- For large systems, the data dictionary grows rapidly in size and complexity.

Data Dictionary (DD) Operators

- $+$: Composition
- $[]$: Selection
- $()$: Optional
- $\{\}$: Iteration
- $=$: Equivalence
- $** **$: Comment

Semantics for DD Operators

- Composite data are defined in terms of primitive data items using following operators:
 - $+$: denotes composition of data items, e.g. $a+b$ represents data a and b .
 - $[, ,]$: represents selection, i.e. any one of the data items listed inside the square bracket can occur. For example, $[a,b]$ represents either a occurs or b occurs.
 - $()$: contents inside the bracket represent optional data which may or may not appear. $a+(b)$ represents either a or $a+b$ occurs.
 - $\{ \}$: represents iterative data definition, e.g.,
 - $\{name\}5$ represents five name data.
 - $\{name\}^*$ represents zero or more instances of name data.
 - $=$ represents equivalence, e.g. $a=b+c$ means that a represents b and c .
 - $* *$: Anything appearing within $* *$ is a comment.

Data Dictionary and Industry Data Governance

- Modern equivalents:
 - API specifications (OpenAPI)
 - Data catalogs
 - Schema registries
- Critical for:
 - Analytics platforms
 - AI/ML pipelines
 - Regulatory compliance

Observation

- DFDs help us create Data model as well as Function model
- As a DFD is refined into greater levels of detail:
 - The analyst performs an implicit functional decomposition.
 - At the same time, refinements of data takes place.
- Represent data that flows between bubbles and not the conditions depending on which a process is invoked.
- Key guidelines,
 - Draw exactly one bubble in context diagram.
 - All external entities should only be represented in the context diagram.
 - In the text for specifying processes, only verbs to be used.
 - External entities should not appear at any other level of DFD.
 - Maximum 7 processes/bubbles per DFD level across all DFD levels.

Observation ... cont'd

- Key guidelines,
 - Each process/bubble should be decomposed to between 3 and 7 processes/bubbles.
 - Not to represent control information in a DFD.
 - Never to represent the order in which different functions are / will be executed. (That is control flow)
 - All functions of the system must be captured in the DFD model - No function specified in the SRS document should be omitted.
 - Only those functions specified in SRS document should be represented.
 - Do not assume extra functionality of the system not specified by the SRS document.
 - Faithful adherence to SRS is critical.

Commonly Made Errors

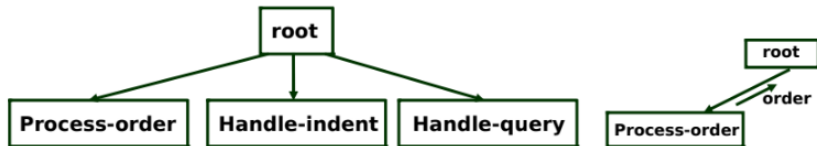
- Unbalanced DFDs
- Forgetting to mention names of the data flows
- Unrepresented functions or data
- External entities appearing at higher level DFDs
- Trying to represent control aspects
- Context diagram having more than one bubble
- A bubble decomposed into too many bubbles in the next level
- Terminating decomposition too early
- Nouns used in naming bubbles

Structured Design

- Aim of structured design
 - Transform results of structured analysis (i.e., a DFD representation) into a structure chart.
- A structure chart represents the system modules.
 - Various modules making up the system,
 - Module dependency (i.e. which module calls which other modules),
 - Parameters passed among different modules.
- Structure chart representation
 - Procedural aspects (e.g, how a particular functionality is achieved) are not represented.
- Main focus of a structure chart:
 - Define the module structure of a software,
 - Interaction among different modules,
 - Easily implementable using programming languages.

Basic Building Blocks of Structure Chart

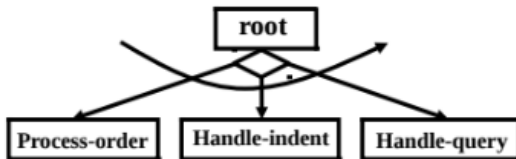
- Rectangular box:
 - A rectangular box represents a module.
 - Annotated with the name of the module it represents.
- An arrow between two modules implies:
 - During execution control is passed from one module to the other in the direction of the arrow. This is the only type of control flow, precisely invocation of a module by another module.
 - Arrows indicate control flow
- Data flow arrows represent:
 - Data passing from module to another in the direction of arrow.
 - Data arrows represent parameter passing



Basic Building Blocks of Structure Chart (contd.)

Quick-sort

- Library modules represent frequently called modules:
 - A rectangle with double side edges.
 - Simplifies drawing when a module is called by several modules.
- Selection (using a diamond symbol)
 - One module of several modules connected to the diamond symbol is invoked depending on some condition.
- Repetition
 - A loop around control flow arrows denotes that the concerned modules are invoked repeatedly.



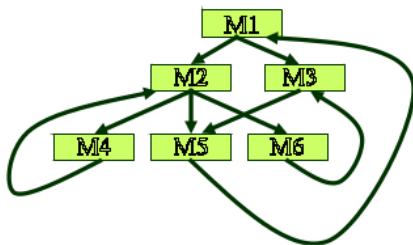
Structure Chart

- There is only one module at the top - the root module.
- There is at most one control relationship between any two modules:
 - if module A invokes module B, module B cannot invoke module A.
- The main reason behind this restriction – consideration of modules in a structure chart being arranged in layers or levels following the standard abstraction principle.
- Principle of abstraction
 - does not allow lower-level modules to invoke higher-level modules:
 - But, two higher-level modules can invoke the same lower-level module.
- Following above principles of abstraction avoids bad design, and is to be mandatorily followed.

Structure Chart - Design Rules

- No cyclic module calls
- Lower-level modules cannot call higher-level modules
- Encourages abstraction and layering

Bad Design



Structured Design and Software Architecture

- Structure charts influenced:
 - Layered architectures
 - Clean Architecture
 - Hexagonal Architecture
- Core idea remains valid:
 - Separation of responsibilities
 - Controlled dependencies

Why Structure Charts Fell Out of Favour?

- Limited support for:
 - Object orientation
 - Concurrency
 - Event-driven systems
- Replaced by:
 - UML component diagrams (to be discussed later)
 - Architecture decision records (ADRs)

Common Industry Failures Linked to Poor Analysis

- Jumping to technology too early
- Missing data ownership clarity
- Ambiguous service boundaries
- Ignoring data lifecycle

Where SA/SD Still Excels

- Large-scale information systems
- Legacy modernization
- Safety-critical and regulated systems
- Early-stage system feasibility studies

SA/SD: Industry Perspective

- Equivalent modern activities:
 - Business process modeling
 - Event storming (conceptual level)
 - System context mapping
- Strength: forces clarity before architecture decisions
- Weakness: not suitable for UI/UX or real-time behavior modeling or low-level performance tuning.

Takeaways for Practitioners using DFDs in Modern Systems

- SA/SD teaches disciplined thinking
- DFDs clarify data responsibilities
- DFDs naturally map to:
 - REST API request/response flows
 - Data pipelines
 - ETL and batch systems
 - Microservice interactions (logical view)
- Particularly effective for:
 - Data-centric systems
 - Compliance-heavy domains
- Modern tools changed — principles did not

DFDs vs UML¹ Diagrams (Industry Reality)

- DFDs answer: **What data moves where?**
- UML answers: **How objects collaborate?**
- In practice:
 - DFDs used early (analysis)
 - UML used later (design)
- They solve different problems

¹Will be discussed in upcoming lectures

Conclusion

- SA/SD remains conceptually relevant
- SA/SD provides a rigorous foundation for system design
- Best used as an analytical lens, not a rigid process
- Strong foundation for architecture and system thinking
- DFDs enable user-centric understanding
- Structured Design bridges analysis and implementation